

GUIDE DE DÉPLOIEMENT

Déploiement sécurisé en production

Une commande pour déployer toute la plateforme sur le PC Serveur Local : TLS sur chaque liaison, secrets et certificats générés automatiquement, services isolés.

Projet	Sentinel-X — Workshop EPSI Bac+4, consortium AetherCorp
Public	Équipe infrastructure et cybersécurité, administrateurs du serveur
Cible	Linux 64 bits (Raspberry Pi OS 64 bits sur Raspberry Pi 5, Debian, Ubuntu) avec Docker
Version	Octobre 2026 — état du dépôt à la date de génération

Sommaire

01 En bref
Ce que fait la commande de déploiement

02 Architecture sécurisée
Services, réseaux, protocoles

03 Prérequis
Matériel, système, Docker

04 Déployer
Installation en quatre étapes

05 Premier accès
Certificat, administrateur, comptes

06 Brancher le matériel
Boîtier USB et webcam

07 Personnaliser
Fichier .env facultatif

08 Mesures de sécurité
Matrice et vérifications réalisées

09 Durcir le serveur
Pare-feu, SSH, mises à jour

10 Exploiter
Supervision, mises à jour, sauvegardes, IA

11 Windows et macOS
Docker Desktop et matériel

12 Dépannage
Problèmes fréquents

1 En bref

```
git clone <dépôt> sentinel-x && cd sentinel-x
docker compose up -d --build
```

Cette seule commande, **sans fichier de configuration**, déploie la plateforme complète :

- une **autorité de certification locale** et un certificat TLS par service ;
- tous les **secrets** : clé de session, jeton du service IA, mots de passe MQTT, mot de passe du premier administrateur ;
- le **dashboard en HTTPS** (le HTTP redirige vers HTTPS), le backend, le broker MQTT, le service IA et la passerelle USB du boîtier ;
- le **compte administrateur initial**, créé au premier démarrage.

Aucune donnée sensible n'est stockée dans le dépôt ni dans un fichier à compléter. Relancer la commande conserve la CA et les secrets existants ; les certificats sont renouvelés automatiquement 30 jours avant leur expiration.

2 Architecture sécurisée

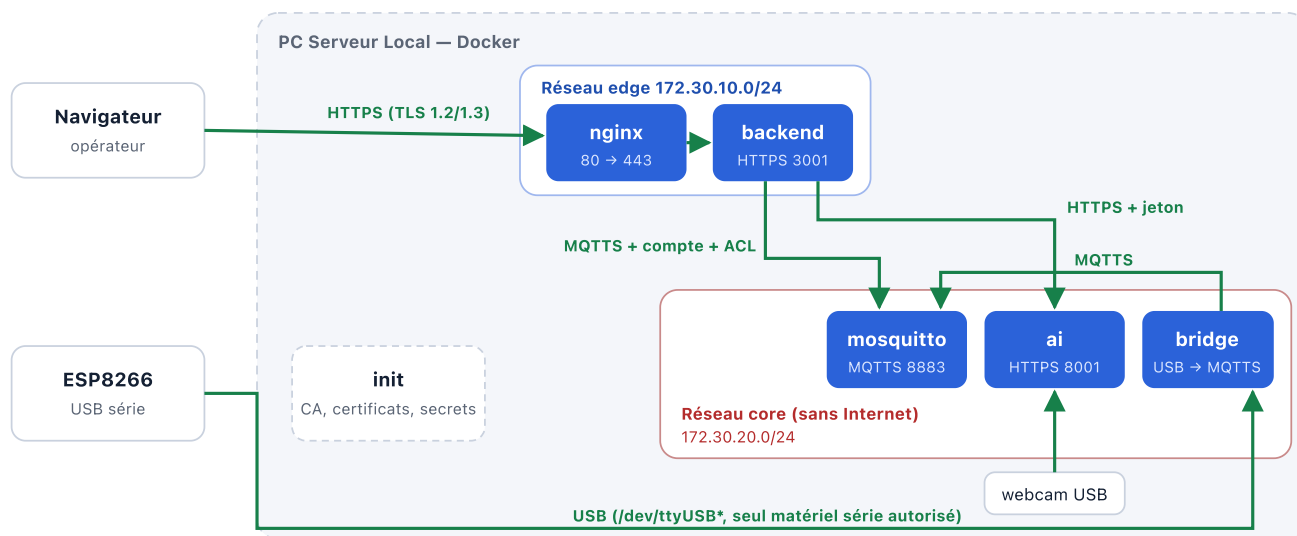


Figure 1. Chaque liaison réseau est chiffrée et authentifiée. Seuls les ports 80 (redirection) et 443 sont publiés.

Service	Rôle	Utilisateur	Exposition
init	Crée la CA, les certificats et les secrets, puis s'arrête	root, sans réseau	aucune
frontend	nginx : dashboard, HTTPS, redirection, relais vers le backend	101	ports 80 et 443
backend	API, authentification, SQLite, temps réel, notifications	1000	aucune (via nginx)

Service	Rôle	Utilisateur	Exposition
mosquitto	Broker MQTT, TLS uniquement	1883	127.0.0.1:8883 seulement
ai	Analyse IA, webcam, reconnaissance faciale	10001	aucune
bridge	Passerelle USB du boîtier vers MQTT	10002	aucune

3 Prérequis

- **Serveur** : Raspberry Pi 5 (4 Go de RAM minimum) sous Raspberry Pi OS 64 bits, ou PC Linux 64 bits (Debian 12, Ubuntu 22.04 ou plus récent).
- **Espace disque** : 10 Go libres (l'image IA pèse environ 2,7 Go).
- **Docker Engine** 24 ou plus récent, avec le plugin `docker compose`.
- **Internet pour la première construction uniquement** : dépendances et modèles (YOLOv8n, YuNet, SFace) sont téléchargés puis vérifiés par empreinte SHA-256. Ensuite, la plateforme fonctionne hors ligne.
- Le boîtier ESP8266 flashé avec le firmware du dépôt (voir le guide du firmware) et, pour la vision, une webcam USB.

```
# Installation de Docker (Raspberry Pi OS, Debian, Ubuntu)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER      # puis se reconnecter
```

4 Déployer

- 1 **Récupérer le projet** sur le serveur :

```
git clone <url-du-dépôt> /opt/sentinel-x
cd /opt/sentinel-x
```

- 2 **Déclarer l'adresse du serveur** (recommandé) : pour accéder au dashboard depuis les autres postes de la table, indiquez son adresse IP ou son nom. Elle sera incluse dans le certificat HTTPS :

```
echo "PUBLIC_HOSTS=192.168.10.1" > .env
```

Sans cette étape, le dashboard n'est accessible que depuis le serveur lui-même (`https://localhost`).

- 3 **Lancer** la plateforme. La première construction dure de 10 à 30 minutes selon la connexion et le matériel :

```
docker compose up -d --build
```

4 Vérifier que tous les services sont démarrés et sains :

```
docker compose ps
```

`init` doit être *Exited (0)* ; `frontend`, `backend` et `ai` doivent être *healthy* ; `mosquitto` et `bridge`, *Up*.

À SAVOIR

Les services redémarrent automatiquement après une coupure de courant ou un redémarrage du serveur (`restart: unless-stopped`), à condition que Docker démarre avec le système (`sudo systemctl enable docker`).

5 Premier accès

5.1 Faire confiance au certificat

Les certificats sont signés par une autorité locale créée au premier démarrage. Le certificat public de cette autorité est exporté dans `deploy/runtime/sentinel-x-ca.crt` . Installez-le sur chaque poste qui consultera le dashboard ; sinon, le navigateur affichera un avertissement de sécurité.

Système	Installation de <code>sentinel-x-ca.crt</code>
Windows	Double-clic → Installer le certificat → Ordinateur local → « Autorités de certification racines de confiance »
macOS	Double-clic → Trousseau Système → ouvrir le certificat → Se fier : « Toujours approuver »
Linux (Chrome)	Paramètres → Confidentialité et sécurité → Gérer les certificats → Autorités → Importer
Firefox (tous systèmes)	Paramètres → Vie privée et sécurité → Certificats → Autorités → Importer
Android	Paramètres → Sécurité → Chiffrement et identifiants → Installer un certificat → Certificat CA

ATTENTION

Ne partagez que `sentinel-x-ca.crt` : c'est un certificat public. La clé privée de l'autorité ne quitte jamais le volume Docker `pki` .

5.2 Se connecter en administrateur

Récupérez le mot de passe généré pour l'administrateur initial (`admin@sentinel-x.local` par défaut) :

```
docker compose exec backend cat /run/sentinel/secrets/initial-admin-password
```

Ouvrez `https://192.168.10.1` (ou `https://localhost` sur le serveur) et connectez-vous.

5.3 Créer les comptes des opérateurs

```
docker compose exec backend node dist/cli/create-user.js operateur@aethercorp.fr "Prénom Nom"
```

Le mot de passe (12 caractères minimum) est demandé deux fois, sans affichage. Créez un compte nominatif par personne : chaque commande et chaque modification de paramètre est journalisée avec l'e-mail de son auteur.

6 Brancher le matériel

Le matériel se branche **à chaud**, avant ou après le démarrage, sans aucune configuration :

- **Boîtier ESP8266** : branchez-le en USB. Le service `bridge` détecte l'unique port USB série (`/dev/ttyUSB*` ou `/dev/ttyACM*`) et transmet les mesures en MQTTS. Débranchement et rebranchement sont gérés automatiquement.
- **Webcam** : branchez-la en USB, puis cliquez sur **Démarrer la webcam** dans la page Caméra du dashboard. Si plusieurs caméras sont présentes, indiquez `VISION_CAMERA_INDEX` dans `.env`.

```
docker compose logs -f bridge      # « Port /dev/ttyUSB0 ouvert » puis « MQTT #... »
docker compose logs -f ai          # état de la webcam et des analyses
```

Les conteneurs n'ont accès qu'aux classes de périphériques nécessaires : ports série pour `bridge`, caméras vidéo pour `ai`, selon des règles de `cgroup`. Aucun conteneur n'est privilégié.

7 Personnaliser

Toutes les valeurs ont un défaut fonctionnel. Pour en changer une, créez un fichier `.env` à partir de `.env.example`, puis relancez `docker compose up -d`.

Variable	Défaut	Rôle
<code>PUBLIC_HOSTS</code>	localhost	Adresses et noms du serveur (liste séparée par des virgules) : certificat HTTPS et origines autorisées
<code>HTTP_PORT</code> / <code>HTTPS_PORT</code>	80 / 443	Ports publiés
<code>ADMIN_EMAIL</code>	admin@sentinel-x.local	Identifiant de l'administrateur initial
<code>READINGS_RETENTION_DAYS</code>	7	Conservation des mesures et analyses
<code>VISION_CAMERA_INDEX</code> / <code>VISION_AUTO_START</code>	0 / false	Choix et démarrage automatique de la webcam
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_SECURE</code> , <code>SMTP_USER</code> , <code>SMTP_PASSWORD</code> , <code>MAIL_FROM</code>	désactivé	Notifications par e-mail (STARTTLS exigé)
<code>MQTT_TLS_PORT</code>	8883	Port du broker sur <code>127.0.0.1</code> , pour une passerelle lancée hors Docker

Après un changement de `PUBLIC_HOSTS`, le certificat HTTPS est régénéré automatiquement au prochain `up`. Le réseau de table n'ayant pas Internet, les notifications nécessitent un serveur SMTP joignable depuis le serveur.

8 Mesures de sécurité

Liaison	Protocole	Authentification
Navigateur → nginx	HTTPS, TLS 1.2/1.3, HSTS ; HTTP redirigé	Session Better Auth (cookie <code>__Secure-</code> , <code>HttpOnly</code> , <code>Secure</code>)
nginx → backend	HTTPS, certificat vérifié par la CA	Réseau interne, IP client de confiance limitée à nginx
backend → mosquito	MQTT (TLS), nom d'hôte vérifié	Compte <code>backend</code> , ACL lecture télémétrie / écriture commandes
bridge → mosquito	MQTT (TLS), nom d'hôte vérifié	Compte <code>bridge</code> , ACL écriture télémétrie uniquement
backend → ai	HTTPS, certificat vérifié	Jeton Bearer partagé
ESP8266 → bridge	USB local	Accès physique au serveur

- **Cryptographie** : clés ECDSA P-256 ; suites ECDHE avec AES-GCM ou ChaCha20 ; TLS 1.0/1.1 refusés ; certificats de 397 jours renouvelés automatiquement.
- **Secrets** : générés aléatoirement (96 bits pour le mot de passe administrateur initial, 192 à 256 bits pour les autres), un propriétaire par fichier, permissions `0400`, lus via des variables `*_FILE`. Ils n'apparaissent ni dans le dépôt, ni dans `.env`, ni dans `docker inspect`. Seul un mot de passe SMTP saisi dans `.env` est passé en variable d'environnement : réservez `.env` à l'administrateur (`chmod 600 .env`).
- **Conteneurs** : utilisateurs non-root distincts, système de fichiers en lecture seule, toutes les capacités Linux retirées, `no-new-privileges`, journaux limités (3 × 10 Mo).
- **Réseau** : broker, IA et passerelle sur un réseau interne sans accès Internet ; backend et IA non publiés.
- **Application** : inscription publique désactivée, mots de passe de 12 caractères minimum, 5 tentatives de connexion par minute et par IP, origines vérifiées. Côté nginx : limites de débit et de connexions par IP, CSP stricte, `X-Frame-Options: DENY`, `nosniff`.

Vérifications réalisées

Le déploiement a été testé depuis une installation vierge, sans fichier `.env`, avec la commande unique de la section 4 :

Contrôle	Résultat
HTTP → HTTPS ; certificat validé par la CA locale	301 puis 200
TLS 1.0 et 1.1 proposés par un client	refusés (alerte « protocol version »)
Accès API sans session / origine étrangère	401 / 403
Force brute avec <code>X-Forwarded-For</code> falsifié	bloquée (429)

Contrôle	Résultat
MQTT anonyme, mauvais mot de passe, en clair, CA non reconnue	refusés
Publication hors ACL (passerelle vers esp8266/led)	refusée (code 135)
Lecture d'un secret d'un autre service	permission refusée
Accès Internet depuis le réseau interne	impossible
Backend (3001) et IA (8001) depuis l'hôte	fermés
Chaîne complète passerelle → MQTTS → backend → IA → dashboard	fonctionnelle
Nouveau up : CA et secrets	conservés

9 Durcir le serveur

Docker protège les services ; le système hôte doit l'être aussi. Ces réglages s'appliquent une fois, en gardant une session ouverte pendant les tests SSH.

1 SSH par clé uniquement : copiez votre clé (`ssh-copy-id`), vérifiez la connexion, puis dans `/etc/ssh/sshd_config` : `PasswordAuthentication no` et `PermitRootLogin no`, puis `sudo systemctl restart ssh`.

2 Pare-feu :

```
sudo apt install ufw
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

3 Mises à jour : `sudo apt install unattended-upgrades`.

ATTENTION

Les ports publiés par Docker contournent les règles UFW. La protection repose donc d'abord sur le fait que le Compose ne publie que 80 et 443, et Mosquitto uniquement sur `127.0.0.1`. N'ajoutez pas de port publié sans en mesurer l'exposition.

10 Exploiter

Supervision

```
docker compose ps      # état et santé des services
docker compose logs -f backend  # journal d'un service
docker stats            # CPU et mémoire en temps réel
```

Mettre à jour

```
git pull
docker compose up -d --build
```

Les données, comptes, certificats et secrets sont conservés dans les volumes.

Sauvegarder et restaurer

Volume	Contenu
<code>sentinel-x_backend-data</code>	Comptes, historique des mesures et des analyses, paramètres
<code>sentinel-x_ai-state</code> / <code>sentinel-x_ai-faces</code>	Modèle d'anomalies entraîné / catalogue facial (données biométriques)
<code>sentinel-x_pki</code> / <code>sentinel-x_secrets</code>	Autorité de certification et secrets

```
# Sauvegarde à froid d'un volume
docker compose stop
docker run --rm -v sentinel-x_backend-data:/data:ro -v "$PWD":/backup alpine \
  tar czf /backup/backend-data.tgz -C /data .
docker compose start
```

Restauration : extraire l'archive dans le volume vide (`tar xzf`), services arrêtés. Les sauvegardes de `pki`, `secrets` et `ai-faces` sont sensibles : chiffrez-les et stockez-les hors du serveur.

Entraîner le modèle d'anomalies

1 Exporter les mesures réelles (base en lecture seule) :

```
mkdir -p entraînement
docker compose run --rm --no-deps -u root \
  -v sentinel-x_backend-data:/backend-data:ro -v "$PWD/entraînement":/export \
  ai python -m app.integrations.backend_client \
  --database /backend-data/telemetry.db --output /export/sensor_data.csv
```

2 Sélectionner les périodes normales dans `entraînement/normal_validated.csv` (voir le guide IA).

3 Entraîner, puis recharger le service :

```
docker compose run --rm --no-deps \
  -v "$PWD/entraînement/normal_validated.csv":/import/normal.csv:ro \
  ai python -m app.anomaly.trainer --csv /import/normal.csv \
  --output /app/state/isolation_forest.joblib
docker compose restart ai
```

Régénérer les secrets

En cas de compromission : `docker compose down`, puis `docker volume rm sentinel-x_secrets` (et `sentinel-x_pki` pour changer d'autorité), et enfin `docker compose up -d`. Toutes les sessions sont invalidées. Les comptes existants sont conservés, et le nouveau mot de passe administrateur ne

s'applique qu'à une base de comptes vide. Après un changement de CA, réinstallez `sentinel-x-ca.crt` sur les postes.

11 Windows et macOS

Docker Desktop fonctionne avec la même commande, mais ne transmet **ni l'USB ni la webcam** aux conteneurs. Le dashboard, le backend, le broker et l'analyse IA des capteurs fonctionnent ; la vision indique « Webcam introuvable ». Pour le boîtier, lancez la passerelle sur l'hôte en MQTTS :

```
# Mot de passe du compte bridge
docker compose run --rm --no-deps --entrypoint cat init /run/sentinel/secrets/mqtt-bridge-
password > bridge-password.txt

.\ai\.venv\Scripts\python.exe scripts\esp_serial_bridge.py --mqtt-host localhost --mqtt-port
8883 `
  --mqtt-ca deploy\runtime\sentinel-x-ca.crt --mqtt-username bridge --mqtt-password-file bridge-
password.txt
```

Protégez `bridge-password.txt` (droits limités à votre compte) et arrêtez le service `bridge` du Compose (`docker compose stop bridge`), inutile dans ce cas. Pour la vision en production, utilisez un serveur Linux.

12 Dépannage

Symptôme	Cause probable	Solution
Avertissement de sécurité du navigateur	CA non installée, ou adresse absente de <code>PUBLIC_HOSTS</code>	Installer <code>sentinel-x-ca.crt</code> ; ajouter l'adresse dans <code>.env</code> puis <code>docker compose up -d</code>
Connexion refusée (403) depuis un autre poste	Origine non déclarée	Ajouter l'adresse utilisée à <code>PUBLIC_HOSTS</code>
Construction de <code>ai</code> interrompue (« Read timed out »)	Réseau lent ou instable	Relancer <code>docker compose build ai</code> : les paquets déjà téléchargés sont en cache
<code>bind: address already in use</code> sur <code>80/443</code>	Autre serveur web actif	L'arrêter, ou changer <code>HTTP_PORT/HTTPS_PORT</code>
<code>bridge</code> : « 0 ports USB plausibles »	Boîtier non branché ou câble de charge seule	Brancher le boîtier avec un câble de données ; vérifier <code>ls /dev/ttyUSB*</code>
Webcam introuvable	Caméra absente, occupée ou mauvais index	Vérifier <code>ls /dev/video*</code> , ajuster <code>VISION_CAMERA_INDEX</code>
Mot de passe administrateur perdu	—	Le relire avec la commande de la section 5.2 (valide tant que le compte n'a pas été recréé)
Aucun e-mail	SMTP non configuré ou injoignable sans Internet	Configurer un relais SMTP joignable ; tester depuis Paramètres